ROOF

Inter-Process Communication Library

Prepared

Luca Grilli

Approved

Luca Pascale	
Date	23/05/2022

Document Issue	
Working Draft	☐
Draft	☐
Proposed Issue	☐
Release Issue	☒

REVISION HISTORY

Issue	Change Description	Issue Date
1.0	Initial issue	27/03/2015
1.1	Paragraph modified: 5.2.2., 5.3.1	23/09/2016
1.2	Added paragraph: PropertyMutator_t Paragraph modified: 4.1.1	27/01/2017
1.3	Added paragraph 5.2.1.1, 5.2.1.2, 5.2.3	18/07/2017
1.4	Modified Table 4	23/05/2022

TABLES OF CONTENTS

1. SCOPE 4
2. APPLICABLE DOCUMENTS 4
3. INTRODUCTION 4
4. ROOF Transport Protocol 5
4.1. Special Messages 7
4.1.1. Control Domain – ROOF Control Protocol 7
4.1.2. Raw Domain – Plain Text Messages 8
5. USING ROOF-IPC library 9
5.1. Messages 11
5.1.1. PropertyMutator_t 11
5.1.2. Generic_text_msg_t 12
5.2. Communication Objects 13
5.2.1. Message Publisher/Subscriber 13
5.2.1.1. Message Publisher 13
5.2.1.2. Message Subscriber 13
5.2.2. Control Port 14
5.2.3. Transport Protocol Parser 16
5.3. Define new messages 17
5.3.1. Control Domain 17
5.3.2. Stream Domain 18
5.3.3. Raw Domain 18
5.3.4. Basic Types Messages (roofipc/NumValueMsg_t.h) 18

LIST OF FIGURES

Figure 1: ROOF-IPC protocol stack 5
Figure 2: Message structure 6
Figure 3: ROOF-IPC protocol stack 8

LIST OF TABLES

Table 1: Applicable documents 4
Table 2: ROOF transport protocol message format 7
Table 3: Communication Objects vs Domains 7
Table 4: PropertyMutator_t extra fields 11
Table 5: PropertyMutator_t byte encoding/decoding 12

Table 6: ControlPort interface.....16
Table 7: TransportProtocolParse usage example 18

1. SCOPE

This document describes the ROOF-IPC Library used for Inter-Process-Communication between applications that use ROOF.

2. APPLICABLE DOCUMENTS

Ref.	Acronym	Title	Originator	Id/Rev

Table 1: Applicable documents

3. INTRODUCTION

ROOF-IPC Library provide the tools needed to exchange "objects" between processes using IP or Shared Memory as transport mechanism. ROOF-IPC makes available the following *"Communication Objects/Patterns"*:

- Publisher
- Subscriber
- ControlPort: Publisher and Subscriber in a single end-point

ROOF-IPC uses the ZeroMQ framework as low-level transport mechanism.

Figure 1**Errore. L'origine riferimento non è stata trovata.** shows the transport protocols involved in the ROOF-IPC library.

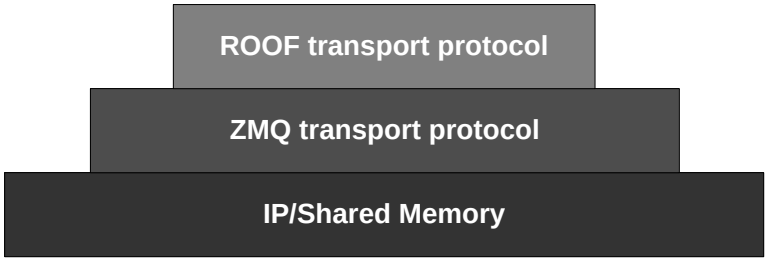

Figure 1: ROOF-IPC protocol stack

ZeroMq framework has its own transport protocol for efficiency and performances reasons (https://rfc.zeromq.org/spec:23/ZMTP/).

For example it manages transparently packet fragmentation but does not manage (serialize/deserialize) in any way the information it is carrying between processes. Basically ZeroMQ provides a pipes system according to the selected communication pattern (Request/Reply, Publisher/Subscriber, Push/Pull, or any combination including message brokers).

The management of the information transported by the ZeroMQ pipes is allowed by the ROOF Transport Protocol.

In this document we will refer to the objects to be exchanged as “Topics” or “Messages” indifferently.

4. ROOF TRANSPORT PROTOCOL

Applications built using ROOF usually handle two different types of messages to be exchanged:

1. **Control messages**: event based messages
2. **Stream messages**: time-based messages

Control messages are triggered by events, for example a button is pressed on the Man Machine Interface. They are used to exchange control information between processes (set/get properties, play/stop, BITE, etc.).

Stream messages are generated (usually) by a Source Block on a time-basis fashion. For example data packets read from a file or from the network. Stream messages can also be generated, for example, by a simulation engine (the source) that build a Stream message for each simulation frame and forward these Stream messages to other simulation blocks/processors.

Within ROOF Control and Stream messages are identify using the concept of *“Domain”*. We will have three possible Domains:

- Stream Domain

- Control Domain
- Raw Domain

Messages have a *"domainID"* field that specify this property.

Messages are composed by an header and a body has shown in Figure 2

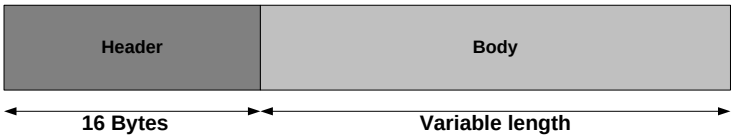

Figure 2: Message structure

The header has a fixed length equal to 16 Bytes. The body has a variable length that depends on the length of the array of bytes generated by the encoding operation of the message.

Table 2 shows the message format defined by the ROOF Transport protocol.

Item	Information	Type	Value Range	Data Length	Offset in Bytes	Description
1.	Destination ID	String	[0,999]	3 Byte	0	Always equal to zero for Publisher and Subscriber (values different from zero will be ingored). In the range [0,999] for ControlPort. A value equal to 0 means that the topic will be sent (and accepted) to (by) each connected communication object (point-to-multipoint). **For ControlPorts AND for topics of the Control Domain**, a value different from 0 means that the topic will be sent to the remote ControlPort having an ID equal to the specified value (point-to-point)
2.	Domain ID	String	[1,2,3]	1 Byte	3	Domain identifier: • 1→Stream Domain • 2→Control Domain • 3→Raw Domain
3.	Message ID	String	[0,9999]	4 Byte	4	Message identifier
4.	Encoding Type	String	[1,2,3]	1 Byte	8	Define the type of encoding used: • 1→Binary • 2→XML • 3→Text (CSV)
5.	Compression	String	[0,1]	1 Byte	9	Define if compression has been used: • 0→no compression • 1→bytes in the message body has been

Item	Information	Type	Value Range	Data Length	Offset in Bytes	Description
						compressed **Note: compression is not actually implemented**
6.	Byte Order Pattern	Binary	0x7FFE	2 Byte	10	Pattern used to check architecture endianness
7.	Unused	String		4 Byte	12	Reserved
8.	Body	String		Variable	16	

Table 2: ROOF transport protocol message format

Not all the Communication Object provided by ROOF are able to manage messages of all the Domains.

Table 3 shows which Communication Object can be used in which Domain.

Item	Communication Object	Stream Domain	Control Domain	Raw Domain
1.	Publisher	Yes	No	No
2.	Subscriber	Yes	No	No
3.	Control Port	Yes	Yes	Yes

Table 3: Communication Objects vs Domains

4.1. SPECIAL MESSAGES

4.1.1. Control Domain – ROOF Control Protocol

Even though a ControlPort is able to manage any kind of message of the Control Domain, the ROOF SDK make available a special message called *"PropertyMutator_t"* that add some extra fields to a control message used to specify which type of action (i.e. SET/GET) the recipient of the message has to carry out. Note that the use of the *PropertyMutator_t* message in conjunction with ControlPorts is not mandatory but is recommended.

Basically a *PropertyMutator_t* is (itself) a Control Message that carry other control messages within its body. Moreover it provides an "operator" field to specify which is the action (SET or GET) for the control message in its body that the recipient of the *PropertyMutator_t* should implement. These considerations applies to the highlighted cell (in blue) of Table 3.

The extra fields provided by the *PropertyMutator_t* message establish the ROOF Control Protocol that extends the ROOF-IPC protocol stack as shown in Figure 3.

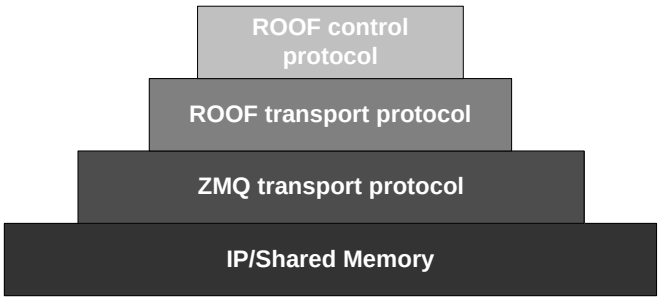

Figure 3: ROOF-IPC protocol stack

Implementation details concerning the PropertyMutator_t message and, in general, about the ROOF Control Protocol will be provided in chapter 5.

4.1.2. Raw Domain – Plain Text Messages

ROOF-IPC makes available another special message that is called *"GenericTextMessage_t"* and is used to exchange raw string messages between processes. *"GenericTextMessage_t"* is a control message. It does not include any operator or any other additional field to specify which kind of action the recipient should take. As a consequence the string command have to be "self-explanatory" and the recipient have to be able "to understand" it.

For example a self-explanatory control message could be (SCPI or SCPI-like style):

SET SYSNAME:MODULENAME:FREQ 100000000

The string is composed by:

- an "operator" (SET)
- a recipient (SYSNAME:MODULENAME)
- a parameter name (FREQ)
- a value to set (100 MHz)

5. USING ROOF-IPC LIBRARY

All the objects to be exchanged via ROOF-IPC must extend/implement the *message_t* interface (jmessage for the Java version).

Description

```
* Common Interface for objects to be exchange between process.
* Every object that has to be exchanged between ROOF processes (via ROOF-IPC)
* must be derived from this class.
* To be successfully sent/received via ROOF-IPC it is mandatory that at least one
* of the pure virtual method for message encoding has to provide a proper message encod-
ing
* implementation
```

Public method

```
* implements encoding to binary format (serialization)
* @param to: string containing the encoded bytes
*/
virtual void ToByteEncoding(std::string& to) = 0;

/**
* implements decoding from binary format (deserialization)
* @param from: string containing the encoded bytes
* @param readIndex: index of the input string to start decoding.
* When  FromByteEncoding return, it must contain the position next to
* the last byte read from the input string
*/
virtual void FromByteEncoding(const std::string& from, int& readIndex) = 0;

/**
* implements encoding to XML format
* @param to: string containing the XML string
*/
virtual void ToXMLEncoding(std::string& to) = 0;

/**
* implements decoding from XML format
* @param from: string containing the encoded XML character
* @param readIndex: index of the input string to start decoding.
* When  FromByteEncoding return, it must contain the position next to
* the last byte read from the input string
*/
virtual void FromXMLEncoding(const std::string& from, int& readIndex) = 0;

/**
* implements encoding to text format
* @param to: string containing the text character
*/
virtual void ToTextEncoding(std::string& to) = 0;

/**
* string containing the encoded text character
* @param from: string containing the encoded text character
* @param readIndex: index of the input string to start decoding.
* When  FromByteEncoding return, it must contain the position next to
* the last byte read from the input string
*/
virtual void FromTextEncoding(const std::string& from, int& readIndex) = 0;

/**
* write message t attributes on an output stream including an header and a footer
* @param xStream: output stream
*/
inline virtual void DumpToStream(std::ostream& xStream);
```

Protected attributes:

```
/// id of the message
unsigned int m_uimessageid;
/// id of the domain of the message
unsigned int m_uidomainid;
/// true/false if ToByteEncoding has been implemented
bool m_bHasByteEncoding;
/// true/false if ToXMLEncoding has been implemented
bool m_bHasXMLEncoding;
/// true/false if ToTextEncoding has been implemented
bool m_bHasTextEncoding;
```

The *message_t* interface has some pure virtual method that is necessary to implement. This methods concern the encoding/decoding operation for 3 formats:

1. Binary
2. XML
3. Text

At least one of the 3 pairs (to/from) encoding/decoding method must provide a working implementation otherwise the object cannot be managed by the ROOF-IPC library. In the next paragraphs will be shown some examples.

5.1. MESSAGES

5.1.1. PropertyMutator_t

Item	Name	type	Description	Note
1.	Source ID	Int	Message originator (Control Port) ID	**A value equal to 0 means sourceID has not been set**
2.	Destination ID	Int	Message recipient (ControlPort) ID	**A recipient have to process a *PropertyMutator_t* message only if *"destinationUID"* is equal to its own ID or if *"destination ID =0"***

3.	Operator	Enum
		{
		SET=1,
		GET
		}
			SET: the recipient will store the control topic contained in the *"msg"* field as its own internal attribute (if existing)
			GET: the originator of the PropertyMutator_t ask for the actual value of the control topic specified by the msgID field.
			The recipient will send a PropertyMutator_t with operator = SET containing the actual value of the control topic specified by "msgID" field	

4.	Encoding type	Enum
		{
		BINARY=1,
		XML,
		TEXT
		}		

5.	Message ID	Int	Unique identifier of the message contained in the *msg* filed	
6.	msg	string	A string containing an encoded message according to the *format* attribute.	Example to be provided

Table 4: PropertyMutator_t extra fields

PropertyMutator_t, that comes with the ROOF-IPC library, is provided with the encoding/decoding functionality in binary format only. As an example the binary encoding/decoding implementation is shown in Table 5.

Byte Encoding

```
voidPropertyMutator_t::ToByteEncoding(std::string& to)
{
    to.clear();

    char* buff = 0;
    int offset = 0;
    int length = 0;
    int iOperator = (int)m_xOperator;
    int iEncodingType = (int)m_xEncodingType;

    Streaming::Put(m_iSourceID, &buff, length, offset);
    Streaming::Put(m_iDestinationID, &buff, length, offset);
    Streaming::Put(iOperator, &buff, length, offset);
    Streaming::Put(iEncodingType, &buff, length, offset);
    Streaming::Put(m_iMessageID, &buff, length, offset);
    Streaming::Put(msg, &buff, length, offset);

    if (buff && offset)
    {
        to.assign(buff, offset);
        delete[] buff;
    }
}
```

Byte Decoding

```
void PropertyMutator_t::FromByteEncoding(const std::string& from, int& readIndex)
{

    if (from.empty())
        return;
    const char* stream = from.data();
    int iOperator     = 1;
    int iEncodingType = 1;
    Streaming::Get(m_iSourceID, stream, readIndex);
    Streaming::Get(m_iDestinationID, stream, readIndex);
    Streaming::Get(iOperator, stream, readIndex);
    Streaming::Get(iEncodingType, stream, readIndex);
    Streaming::Get(m_iMessageID, stream, readIndex);
    Streaming::Get(msg, stream, readIndex);
    m_xOperator = (PropertyMutatorOP)iOperator;
    m_xEncodingType = (EncodingType)iEncodingType;
    if (m_pmsg != nullptr)
    {
        delete m_pmsg;
        m_pmsg = nullptr;
    }
}
```

Table 5: PropertyMutator_t byte encoding/decoding

Streaming::Put and Streaming::Get, that came with ROOF, write/read data to/from memory. They are, basically, the same as a “memcpy”.

5.1.2. Generic_text_msg_t

To be added

5.2. COMMUNICATION OBJECTS

ROOF-IPC provides the following communication objects:

- Message Publisher
- Message Subscriber
- Control Port

5.2.1. Message Publisher/Subscriber

5.2.1.1. Message Publisher

To be added

5.2.1.2. Message Subscriber

A Message Subscriber is able to manage Stream domain messages only. A Subscriber can subscribes to receive all messages of the Stream domain (default behaviour) or a list of specifics messageID using the "setFilterList()" method. A Message Subscriber can subscribe to receive messages from one or more Message Publisher. Message Subscriber offers two possibilities to process the incoming messages:

1. Boost Signal/Slot
2. Re-implementing "handleMessage()"

Public Method

```
/**
 * method to be used to connect a Boost slot to the signal emitted
 * when a message is received. The slot will be executed in a separate thread
 * @param xMessageReceivedSlot Boost slot function
 */
 void connectToMessageReceivedSignal(const MessageReceivedSlot& xMessageReceivedSlot);

/**
 * set address to subscribe to
 * @param strAddress address of the Message Publisher to subscribe to
 */
 void setAddress(const std::string& strAddress);

 std::string address() const;

/**
 * set a list of addresses to subscribe to
 * @param xAddressList list of addresses to subscribe to
 */

 void setAddressList(const std::vector<std::string>& xAddressList);
 std::vector<std::string> addressList() const;

/**
 * set the list of messageID to subscribe to. This object will receive
 * a message only if its messageID is contained xMessageIdList
 * @param xMessageIdList list of messageID to subscribe to
 */
 void setFilterList(const std::vector<unsigned int>& xMessageIdList);
 FilterList filterList() const;
```

```
/**
 * call back called once a message is received. Default implementation
 * is empty. Can be re-implemented by the user to process a received message
 * @param strMessage encoded message
 */
 virtual void handleMessage(const std::string& strMessage);
```

5.2.2. Control Port

A Control Port is able to manage messages of all the defined domains. For messages of the Stream and Raw Domain a Control Port has the same behaviour of a Publisher and basically publishes messages that will be received by every communication object that is subscribed to it.

For messages of the Control Domain (basically any message sent via a PropertyMutator_t) the Control Port makes available some additional features:

- It is possible to set a destination ID for a message so that it will be received only by the Control Port having that destination ID
- It is possible to set filters based on:
  - Domain ID
  - Message ID

Using filters it is possible to define which specific messages will be received by a ControlPort. The messages that meet the filter criteria will be received. The messages that do not meet the filter criteria will not be sent by the Source Control Port. It means that filtering is performed at the publisher side and as a consequence filters limit the network traffic if properly used.

Most important methods of the ControlPort class are shown in Table 6.

Description

```
 * ControlPort is one of the communication objects of the ROOF-IPC library.
 * It can be used to send/receive messages between processes using IP protocol or
 * shared memory. It is based on a publisher/subscriber pattern and as a consequence
 * it is possible to subscribe a ControlPort to one or more remote ControlPorts
 * and it is also possible to subscribe to specific messages only (Filters).
 * ControPort manages messages of all the existing domains (see ROOF-IPC doc).
 * Subscription can be configured at domain level only or at <domain,message> level.
 * More than one filter can be added to a ControPort to properly receive only the
 * requested messages.
 * ControlPort implements a point-to-multipoint communication schema. For messages
 * of the Control Domain managed via PropertyMutator_t it provides a point-to-point
 * schema specifying the ID of the destination ControlPort
```

Public methods

```
/**
* Default constructor
```

```
* @param uiId: controlport ID
*/
ControlPort(unsigned int uiId);
virtual ~ControlPort();

/**
* @return the controlport ID
*/
unsigned int id() const;


/**
* When a controlport received a new message a (Boost) signal is emitted.
* This function connect that signal to the specified (Boost) slot.
* The slot must take a std::string at its input and perform the parsing of
* the string using the Protocol Parser (see TransportProtocolParser.h)
* @param xMessageReceivedSlot: the slot(callback) function address
* to trigger when the controlport receive a message. The argument of the function
* must be an std::string (see MessageReceivedSlot definition)
*/
void connectToMessageReceivedSignal(const MessageReceivedSlot& xMessageReceivedSlot);

/**set the end-point string of the controlport
* @param strAddress: end-point address
*/
void setAddress(const std::string& strAddress);

std::string address() const;

/**
* set a list of remote controlport end-point
* @param xRemoteAddressList: list of end-point
*/
void setRemoteAddressList(const std::vector<std::string>& xRemoteAddressList);
std::vector<std::string> remoteAddressList() const;

/**
* Add a subscription filter. It will be received only messages that
* has a domainID = uiDomainId and a messageID = iMessageID
* @param uiDomainId: id of the requested message domain (see message_defines.h)
* @param iMessageId: id of the requested message. If iMessageId = -1
* the filter will take into account the uiDomainId only
*/
void addFilter(unsigned int uiDomainId, int iMessageId);

/**
* Same as addFilter() called recursively on a list of pairs  <DomainId, MessageId>
*/
void setFilterList(const std::vector<std::pair<unsigned int, int>>& xDomainIdMessageId-
PairList);

/**
* start the controlport
* @return: true if the controlport has been correctly started
*/
bool start();

/**
* stop the controlport
*/
void stop();

/**
* @return: true if the controlport has been correctly started
*/
bool isRunning() const;

/**
* push data (to be sent) in the controlport
* @param pmsg: message_t to be sent
*/
```

```
void pushData(message_t* pmsg);
```

Table 6: ControlPort interface

Note: when a ControlPort is instantiated it has an empty string filter defined (“ “). It means that it will receive every message sent (using ROOF transport Protocol) independently by the destinationID, domainID and messageID.

5.2.3. Transport Protocol Parser

The TransportPortocolParser is the object that allow to retrieve all necessary information to process the incoming messages. It provides:

- encoding type (Binary, XML,…)
- domainID of the message
- ID of the message
- destinationID of the message (it applies for Control messages only)

Refer to *“roofipc/TransportProtocolParser.h”*

5.3. DEFINE NEW MESSAGES

5.3.1. Control Domain

To define a new message of the Control Domain follow these steps.

1. Implement the new message (*NewMessage_t*) extending the message_t class
2. Set *NewMessage_t ::m_uidomainid = ROOF::ipc::Control*
3. Define a new messageID and set *NewMessage_t ::m_uidomainid = messageID*. It is important that the defined messageID is unique within the Control Domain. Due to the fact that message IDs can be defined in different files it is very important to be sure that the defined ID is unique otherwise decoding of *NewMessage_t* will fail.
4. Set *"m_iDestinationID"* as requested

For the sending side:

1. Once the ControlPort is properly configured and started (start()), call *ControlPort::pushData(NewMessage_t)*

For the receiving side:

1. Parse the received string using TranportProtocolParser

Transport Protocol Parser (generic) usage example

```
ROOF::ipc::TransportProtocolParser xTPParser(strMsgString.c_str(), strMsgString.size());
unsigned int uiMessageId = xTPParser.messageid();
ROOF::ipc::EncodingType eEncodingType = xTPParser.encoding();
std::string strMessageBytes = std::string(xTPParser.messageBytes(), xTP-
Parser.messageBytesLength());

if (uiMessageId == GENERIC_TEXT_MSG)
{
    ROOF::ipc::messages::generic_text_t xGenericText;
    int offset = 0;
    xGenericText.FromTextEncoding(strMessageBytes, offset);
}
else if (uiMessageId = PROPERTYMUTATOR_MSG)
{
    ROOF::ipc::messages::PropertyMutator_t propMutator;
    int offset = 0;
    propMutator.FromByteEncoding(strMessageBytes, offset);
}
else if (uiMessageId = NEWMSG_MSG)
{
    NewMessage_t nMsg;
    int offset = 0;
    if(eEncodingType == BINARY)
        nMsg.FromByteEncoding(strMessageBytes, offset);
    else if(eEncodingType == XML)
        nMsg.FromXMLEncoding(strMessageBytes, offset);
    else if(eEncodingType == TEXT)
        nMsg.FromTextEncoding(strMessageBytes, offset);
```

```
    else
        //error condition

    //do something(nMsg)
}
```

Table 7: TransportProtocolParse usage example

5.3.2. Stream Domain

Same as for Control Domain messages but:

NewMessage_t ::m_uidomainid = ROOF::ipc::Stream

5.3.3. Raw Domain

Same as for Control Domain messages but:

NewMessage_t ::m_uidomainid = ROOF::ipc::Raw

5.3.4. Basic Types Messages (roofipc/NumValueMsg_t.h)

Independently from the domain, in case it is necessary to send messages formed by a single basic type (int, float, double) ROOF makes available a template class that includes binary encoding/decoding. Using this class is just needed to specify the template type, a messageID and a domainID to build a new message.

The class is called “NumValueMsg_t” and some class has been predefined for the following basic types:

- Int (IntValueMsg_t)
- Long (LongValueMsg_t)
- Float (FloatValueMsg_t)
- Double (DoubleValueMsg_t)

To define a new message that has a single attribute of type equal to one of the previously described ones it is only needed to assign a new messageID and a domainID.

For example suppose we want to build a new message of the Control Domain to set/get the send interval of a heartbeat message to/from a remote process. We just need:

```
typedef IntValueMsg_t(newID, ROOF::ipc::Control) heartBeatPeriod_t ;
```

The new message is ready to be used.